



Dynamic Neighborhood Selection for Context Aware Temporal Evolution Using Graph Neural Networks

Muhammad Ali Zeb¹ · M. Irfan Uddin¹ · Ala Abdulsalam Alarood² · Muhammad Shafiq³ · Safa Habibullah⁴ · Abdulkream A. Alsulami⁵

Received: 31 May 2024 / Accepted: 4 November 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Graph neural networks (GNN) have seen significant growth recently for modeling temporal evolution in dynamic networks. Representation of complex networks in the form of graph data structures has enabled researchers to study how entities within these networks interact with each other. These interactions evolve over time. Developing a generic methodology for modeling this temporal evolution in complex networks for tracking evolving relationships has been a significant challenge. Most of the existing methods fail to extract contextual representations of historical neighborhood interactions for future link prediction. To address these challenges, this paper presents a novel method for modeling temporal evolution in complex networks using GNNs. A Context-Aware Graph Temporal Neural Network (CATGNN) method that uses dynamic neighborhood selection based on common neighbors for a given node is presented. The method uses dynamic neighborhood selection using contextual embeddings extracted from the historical interactions of the down-sampled set of neighbors of a central node based on a common neighborhood. Fixed-sized contextual memory modules are constructed for each node that store the historical interactions of its neighbors and are updated based on the recency and significance of interactions. The proposed method has been evaluated using six real-world datasets and has comparable performance against state-of-the-art methods, both in terms of accuracy and efficiency. It shows an improvement of 7.52 to 0.05% over the baselines in terms of average precision. The results demonstrate that the proposed CATGNN model can capture complex patterns of change that are difficult to identify using traditional techniques by propagating information over the graph structure. The model can be applied in various fields involving complex systems.

Keywords Complex networks · Context-aware temporal graph neural network · Graph neural networks · Neighborhood selection · Temporal graph networks

✉ M. Irfan Uddin
irfanuddin@kust.edu.pk

Muhammad Ali Zeb
alizeb@kust.edu.pk

Ala Abdulsalam Alarood
aasoleman@uj.edu.sa

Muhammad Shafiq
shafiq@kust.edu.pk

Safa Habibullah
shhabiballah@uj.edu.sa

Abdulkream A. Alsulami
Aaalsulami6@uj.edu.sa

¹ Institute of Computing, Kohat University of Science and Technology, Kohat 26000, Pakistan

² College of Computer Science and Engineering, University of Jeddah, 21959 Jeddah, Saudi Arabia

³ Institute of Numerical Sciences, Kohat University of Science and Technology, Kohat 26000, Pakistan

⁴ Department of Information Systems and Technology, College of Computer Science and Engineering, University of Jeddah, Jeddah, Saudi Arabia

⁵ Department of Information Technology et al-Kamil, University of Jeddah, Jeddah, Saudi Arabia

Introduction

Representation of complex networks in the form of graph data structures has motivated researchers to study how entities within these networks interact with each other. These interactions evolve over time. Developing a generic methodology for modeling this temporal evolution in complex networks has been a challenge.

Temporal relationships between different objects or entities in a network can be modeled as topological relationships that change or evolve over time while preserving their essential shape or structure [1]. In other words, it refers to the study of how the topology of a space changes over time. Generalized formal models are required for the representation of relations among entities connected through time and context forming complex networks. Entities in a network might be connected through space, occurring in the same location, while the relation among the entities might evolve over time [2]. Entities in a network might also be related through time dependency in a specific time frame. For example, users expressing their opinions on social media about an issue, the effect of exogenous events on the adaptation of user sentiments, users showing interest in a news story in a specific duration, and customers from a particular area showing interest in an item in a specific season. The temporal significance of relationships among entities forming evolutionary networks cannot be ignored.

The real world has numerous examples of non-linear representations of entity relationships in their respective environments. These representations and their interactions have been studied by researchers for decades [3, 4]. Graph neural networks (GNNs) are neural network architectures specifically designed to work with graph-structured data [5, 6]. With the recent advancement of GNNs, there is renewed interest in developing models that evolve with

time [7–9]. In [10] a detailed survey of the different GNNs has been presented that can be used for different graph types including static and dynamic graphs. They also distinguish between the discrete-time and continuous-time dynamic graphs. A change in the network is considered to have occurred when (i) there is a change in the features of the nodes (entities) at different time intervals, (ii) change in the interaction between two nodes, and (iii) appearance or disappearance of entities in the network. Figure 1 shows a toy example of temporal evolution in a network between timestamps t_0 and t_1 and link prediction over the subsequent timestamp t_2 . Traditional representation of nodes in temporal networks results in non-contextual aggregation of messages from neighboring nodes. On the other hand, context-aware node embeddings based on common neighborhood and their historical interactions provide combined structural representations for enhanced link prediction. We can explore the dynamic behavior of entities in a network, which might evolve in response to exogenous events, because of drift in user interests, or maintaining semantics between two given entities with sequential representation of their properties within a large non-linear environment. Although each of these environments warrants a specialized research focus, a generalized model for the representation of these complex networks can be developed.

In this work, we propose a Context-Aware Temporal Graph Neural Network (CATGNN) for the representation and adaptation of these properties at the local and global levels in a graph structure using GNN. This representation allows us to manipulate the temporal properties using common neighborhood and hierarchical relationships at the sub-graph level. The objective is to maintain the context with the evolution of relations among the entities of the network.

The novelty of the CATGNN is the contextual representation of a common neighborhood by maintaining

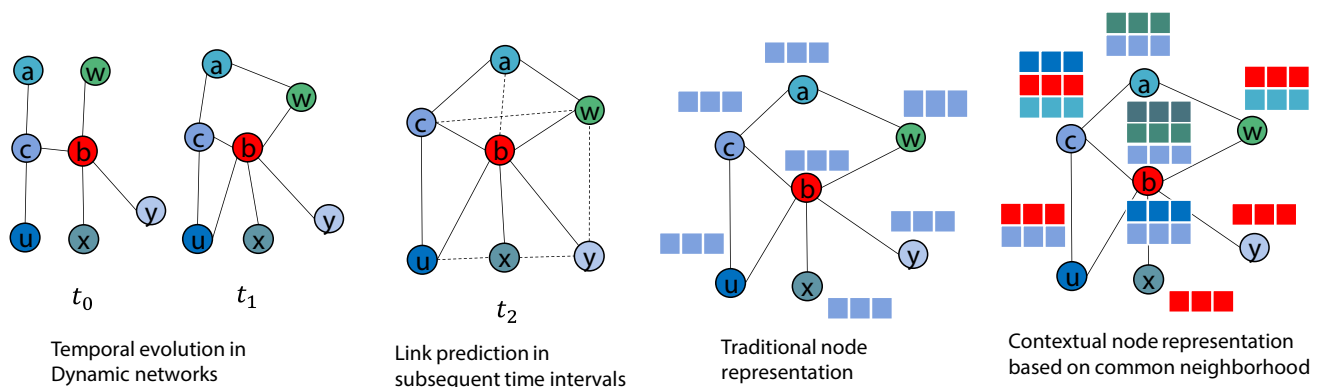


Fig. 1 A toy example of temporal evolution in a network between timestamps t_0 and t_1 and link prediction over the subsequent timestamp t_2 . Traditional representation of nodes in temporal networks results in non-contextual aggregation of messages from neighboring

nodes. On the other hand, context-aware node embeddings based on common neighborhoods and their historical interactions provide combined structural representations for enhanced link prediction

down-sampled data structures of the historical interactions of each node with its neighbors. Fixed-sized memory modules (tensors) are used to efficiently store the nodes with historical interactions with a target node and aggregate the edge features between the node pairs (target node with each neighboring node) along with the temporal encoding of the interactions. These memory modules are updated based on the recency of interactions. Attention scores are used to identify temporally significant neighbors. The main objectives of the proposed study are outlined as follows:

1. To explore GNNs to develop an integrated model for the representation and propagation of the relationships among entities in complex networks.
2. To study the resulting graph structures at different levels of granularity for maintenance of semantics and contextual details due to evolving temporal relationships between the entities of the graphs.
3. To illustrate the generalization and adaptation capabilities of the proposed model by performing experiments using real-world data that can be represented through graph structures.
4. To evaluate and compare the performance of the proposed GNN-based modeling against state-of-the-art approaches.

The remaining article is structured into the following sections: Section 2 provides a literature review of the existing studies conducted on GNN-based techniques. The proposed GNN-based methodology uses dynamic neighborhood selection based on attention scores of significant neighbors for a given node. The methodology is explained in detail in Section 3. Section 4 discusses a detailed analysis of the experimental results, including a comparative evaluation with state-of-the-art methods. Finally, Section 5 concludes the article by providing a brief overview of the proposed work as well as acknowledging the limitations and suggesting future research directions.

Literature Review

GNNs have been used and studied in various areas recently such as sentiment analysis [11, 12], program analysis [13], named entity recognition and relation extraction [4, 14], and recommendation systems [15]. Similarly, temporal context has been studied in various fields using GNNs. Pioneering work has been performed by [16] to study the evolution of a social network with time. Temporal adaption of aspect-based sentiment analysis has been studied by [17, 18]. Other areas that have used GNNs for capturing the temporal context include traffic prediction [19], link prediction [20], recommender systems [21, 22], event-based data [23], studying

emotion perceptions [24], exploring neighbor relationship filtering [25], and multivariate time series forecasting [26].

Neighborhood selection and efficient message passing have been of particular interest to researchers studying temporal graph neural networks. To prevent excessive assimilation of node embeddings across different types, H. Peng et al., [27] employed a label-aware neural similarity measure for neighbor selection based on node attributes. Their proposed model reinforced the relation-aware mechanism. The model selected similar neighbors within a relation before aggregating information from various relations to generate node embeddings. Then, they proposed a scalable reinforcement learning method for efficient neighbor selection in multi-relational graphs. The model generated more discriminative and explainable embeddings by recognizing relation importance through a filtering threshold. Moreover, X. Zang and B. Tang [28] introduced DGNC, a DGE approach that integrates both neighborhood and community perspectives. The neighborhood view captures node embeddings based on timestamped interaction events, modeling local graph variations with a time-varying attention vector. Meanwhile, the community view encodes node and community features to capture global graph evolution through community correlation. Cross-supervised contrastive learning was employed to integrate these views, facilitating collaborative guidance and exchange of information, and enhancing graph embedding quality.

In addition, Zhu et al. [29] introduced the DGTCC model, a continuous-time dynamic graph framework that mines the latent evolution trend of graph representation by integrating topology and temporal information. In this approach, a self-attention-based temporal encoder creates the node representation first. It does this by calculating neighbor node importance weights in temporal sub-graphs and storing the results in the contextual memory module. The model uses a contrastive learning approach to maximize the mutual information of a node that happened in two close temporal views after sampling the node representation from the memory module. This helps to follow the node's evolutionary tendency. M. Zhang et al. [30] suggested a technique called personalized GNN with Attention Mechanism GNN using the Reddit dataset. In contrast to the traditional GNN model, which considers the role of each user when updating the node embedding, A-PGNN primarily consists of two components: the Personalized GNN was used to extract the personalized structural information in each user behavior graph. Explicitly modeling the impact of previous sessions on the current session, the Dot-Product Attention method utilizes the Transformer net.

Another interesting research study was carried out by Y. Li et al., [31] who introduced a novel approach named the T-TGNN for learning continuous dynamic network representations using Reddit posts. The aim is to assimilate comprehensive event-driven data within continuous dynamic networks. Initially, they employed ordinary neural differential equations

to capture the ongoing temporal evolution in dynamic networks. Following this, they integrated transformer mechanisms to amalgamate both temporal and structural insights. To enhance the capability of the transformer in aggregating the effects of substantial regional changes, they developed a spatial event coefficient utilizing an attention mechanism to depict the global scope of event interdependencies in dynamic networks. F. Wang et al. [32] proposed a model that uncovers a key mechanism driving community evolution: individual temporal semantics or changing subjects. It is suggested to use a novel dynamic community detection model that considers natural evolutionary traits. First, social networks are divided into snapshots by the model. Then, using information from the previous snapshot and particular subjects that change, it detects the community structure at each snapshot. Ultimately, the development of users interested in subject distributions and distributions about communities topics are determined. Z. Mu et al. [33] built a Contrastive Hawkes Graph neural network with Dynamic Sampling (CHGDS) to explore varied neighbors and model explicit historical importance in the case of a continuous-time dynamic graph. By using Hawkes graph convolution to determine the time effect of nodes, CHGDS can differentiate between the impact of various historical events on the node in question. Their proposed approach blends location and time exploration of multi-hop neighbors with dynamic sampling.

With the increased adaptation of GNNs for temporal network data, efficient training of large-scale dynamic graphs has been a focus of research recently. Gao et al. [34] have proposed a batching mechanism for the efficient training of large-scale dynamic graphs. Gao et al. [35] have presented a dynamic data displacement method for efficient training of temporal GNNs. Zhang et al. [36] have used temporal contextual embedding based on previous interactions for adding potential edges for better learning of downstream tasks. Transformers have also been incorporated for efficient training and scalability of dynamic graphs [37–39].

Neighborhood aggregation for node feature construction is a widely studied area, and some excellent methods have been proposed [40] for neighborhood selection. However, the focus of the current study is temporally evolving networks where the edge relation between interacting nodes might change over time. Some researchers have also highlighted when the node aggregation may or not work [41]. This study assumes that the relationship between entities in a network might evolve with time and is evaluated on temporally evolving graph datasets. Furthermore, the structural and computational complexity of constructing and maintaining contextual embeddings for link prediction in large temporal networks with evolving node interactions is an active research area requiring efficient and generalized solutions.

We have built on the idea that using a few influential temporal neighbors for computing a central node's

representation might be possible without compromising the predictive performance [42]. Identification of influential neighboring nodes in graph networks has been extensively studied by researchers. Ghafouri et al. [43] have presented a detailed discussion of influence maximization among neighboring nodes in a network. They have utilized the generation of synthetic graphs to identify influential individuals in social networks through probabilistic link prediction. Developing a generalized method appropriate for tracking the evolving relationships among entities in complex networks over time is a significant challenge. Most of the current approaches have applied GNNs to study graph-structured temporal data using the snapshot-based temporal graph approach that follows the time-series format. Real-world complex topological systems with heterogeneous entities and relationships, however, might evolve at different levels of granularity. Therefore, a context-aware dual temporally adaptive mechanism, which can model the feature-dynamic, as well as neighborhood selection with temporal evolution of the scalable graph-structured representation of these complex systems is suggested. The novelty of the proposed model is maintaining a combined neighborhood representation that keeps track of the historical interactions between nodes. This representation is dynamically updated based on an attention mechanism selecting significant nodes aggregating the timestamps and link features between the selected set of neighbors. This allows for parallel neighborhood representation of neighboring nodes at different levels of granularity, by discarding the insignificant nodes from computation, promising efficient link prediction. Table 1 shows an overview of the literature review, highlighting the main method, domain, features, and limitations of the existing studies.

Methodology

One of the limitations of applying traditional GNN approaches to evolving temporal networks is the use of single vector feature representation. Figure 1 shows the vector representation of node features in the traditional GNN models. The features for a target node are aggregated from the neighboring nodes. This approach can lead to the problems of over-squashing and over-smoothing [44]. Figure 2 shows the modeling of the link prediction based on contextual feature representation exploiting the common neighborhood and the change in feature representations of the nodes at different time intervals. At 0-hop the self-representation of each node is shown (Fig. 2 Left). Contextual feature representation for each node is obtained using the common neighborhood for the d-hop neighborhood. The value of d is passed as a parameter to the model. Figure 2 (center) shows the 1-hop representation for each node (except for node 1, which has a 2-hop neighborhood representation).

Table 1 Summary of literature review

Reference	Domain	Method	Features	Limitations
Chen et al., [11]	Recommender Systems	CT-GNN, GNN-based session modeling	Captures item relations via graph structure, uses temporal data, improves recommendation diversity with Local Item Representation Learning	Limited use of temporal information in current recommender systems
Li et al., [12]	Knowledge Hypergraph Representation	Hypergraph Joint Embedding model	Applies 3D and 2D convolution for efficient knowledge extraction, entity mask mechanism speeds training	Shallow models limit extraction, deep models lead to excessive parameters and complexity increases
Gong et al., [13]	Multimodal Named Entity Recognition (MNER)	GNN-MNER	Uses GNN for fine-grained text-image alignment, improving interaction between modalities	Struggle with semantic mismatches and lack of detailed correspondences
Li et al., [14]	Dynamic GNN	Ada-DyGNN	Reinforcement learning model for adaptive node updates, robust against noise in dynamic graphs	Lack of control over which nodes to update, leading to potential propagation of noisy information
Bai et al., [17]	Social Recommendation	Temporal Enhanced GNN	Apply temporal strength information and model user preferences based on basic preferences and social influence	The model may not fully address the complexity of dynamic social influences over time
Fan et al., [18]	Recommendation	Temporal Collaborative Transformer (TCT) layer	To capture collaborative signals and temporal dynamics, propagating information over a continuous-time bipartite graph	It can be inefficient to effectively scale with highly dynamic datasets or complex user-item interactions
Guo et al., [19]	Event-Based data	CTGN	Models both link timestamps and durations, with ODE parameterized by a GNN	May struggle with modeling continuous dynamics over large-scale graphs
Peng et al., [8]	Neighborhood Selection	GNN	Constructs transaction subsets, uses a similarity measure based on random walks, and uses reinforcement learning for neighbor selection	May experience challenges with scalability and computational efficiency when handling large transaction datasets
Zang and Tang [21]	Dynamic Graph Embedding and dynamic graph analysis	Dynamic GNN	Neighborhood view for local variations, community view for global evolution, and contrastive learning for integration	Focus mainly on local structures, ignoring global community structures
Zhu et al., [22]	Dynamic Graph Representation	Dynamic GNN with contrastive learning	Temporal encoder with self-attention, contextual memory module, and mutual information maximization for evolution tracking	May struggle with sparse temporal data and handling large-scale dynamic graphs efficiently
Zhang et al., [23]	Session-Aware Recommendations	Personalized GNN	To extract structural information and a Dot-Product Attention mechanism to model historical session effects	This method may struggle with scalability and efficiency when dealing with large user behavior graphs
Li et al., [24]	Continuous Dynamic graph representation	T-TGNN	Combines temporal changes with global event-driven information using spatial event coefficients and attention mechanisms	May face challenges in efficiently modeling large-scale networks due to the computational complexity of transformers

Table 1 (continued)

Reference	Domain	Method	Features	Limitations
Wang et al., [25]	Community evolution	Individual temporal semantics in social networks	Partitions social networks into snapshots, utilizes individual temporal semantics, and tracks topic distribution evolution	The model may face challenges in scalability and computational efficiency when processing large social network datasets
Mu et al., [26]	Event prediction	Contrastive Hawkes GNN with Dynamic Sampling	Modelling historical importance with Hawkes graph convolution explores diverse neighbors and employs contrastive learning for robustness	Inefficiency when processing large and complex dynamic graphs
Gao et al., [1]	Training and Efficiency	ETC	Generic framework for efficient training of T-GNNs	Effectiveness may vary with graph structures
Gao et al., [3]	Training and Efficiency	SIMPLE	Addresses CPU-GPU data loading bottleneck and dynamic data placement mechanism	Effectiveness may vary with graph structures
Zhang et al., [4]	Graph structure learning	Time-aware Graph Structure Learning (TGSL)	Predicting context embeddings and using Gumbel-Top-K for edge selection	Performance depends on data quality and complexity in joint learning
Wu et al., [5]	Dynamic Graph Modeling	Simple Transformer model customized for dynamic graph modelling	Leverages Transformer's self-attention for long-term dependencies and simple design	May overlook complex temporal dynamics and structural dependencies
Yu et al., [6]	Dynamic Graph Learning	Transformer-based architecture for dynamic graph learning (DyGFormer)	Employs Transformer with neighbor co-occurrence encoding and sequence patching for longer history modeling. Includes DyGLib for reproducibility	Inconsistent baseline results across datasets may arise from varied implementations
Cong et al., [7]	Scalability	Dynamic Graph Learning using Transformer architectures	Uses a TGN and subgraph-based node sampling; incorporates self-supervised pre-training tasks for improved generalization	May face challenges with computational efficiency when applied to highly variable graph datasets
Peng et al., [8]	Neighborhood selection	Reinforced GNN	Adaptive neighbor selection enhances explainability	Computational complexity increases with graph size, potentially affecting scalability
Xie et al., [9]	Neighborhood aggregation	Adaptive-layer GNN	Node-specific aggregation improves performance	Less effective when neighbors are too similar or dissimilar
Finkelshtein et al., [27]	Cooperative GNN	Explainability of GNN	Initializes and iteratively refines importance scores for both input features and hidden neurons for better explanations	The proposed model may face challenges with computational efficiency and complexity when applied to very large GNNs

This is intended for illustrative purposes only. The value of d is uniform for the neighborhood selection of each node. Future links are then predicted using the common neighbors between two given nodes (Fig. 2 Right).

The CATGNN model based on GNN is proposed in this paper. The model extends the standard Temporal Graph Networks (TGNs) [16]. TGNs have been successfully applied in various applications such as traffic prediction, social network analysis, and recommendation systems, as discussed in the literature. Contextual information is incorporated using a context-embedded weight matrix for capturing temporally evolving relationships between different entities in a network. CATGNN model incorporates context-embedded time-dependent features that adapt to the temporal changes in the graph structure into temporal vectors at different time intervals (explained in detail in Section 3.2). This allows for the modelling of temporal dynamics in graph data and enables the prediction of future relationships (links) between the entities of the graph. Node features at different layers of a GNN rely on the aggregation of features from neighboring nodes, called message passing. The emphasis of the proposed method for incorporating contextual adaptation is due to the dependency of message passing between the nodes in GNN on its neighbors. However, message passing from all the neighboring nodes can be inefficient and time-consuming. Neighborhood selection is therefore an important step in the aggregation of useful features for a given node at each layer of a GNN. It becomes more important when studying temporally evolving networks. The neighbors of a node might evolve over time and continuously evolving neighbors with context adaptation might be true representatives for message passing between the neighboring nodes. CATGNN model, therefore, focuses on the selection of

temporally significant neighbors that are common between the source and destination nodes at a given time. The method is explained in the following sections.

Graph Representation

A given Graph G_t at time $t \in T$ is processed into a set of embeddings M_t . A weighted directed graph $G_T(V, E)$ is then constructed where the nodes (represented by V) are connected through a set of edges E over a given time domain T , where the number of nodes is represented by node embeddings generated. In a dynamic graph, the edges are the interactions, current or historical, among the nodes of the graph.

Dynamic Neighborhood Selection for Link Prediction

Node representation learning is performed to extract graph nodes from the given domain. Neighboring nodes are closely occurring groups of similar nodes that can be combined to form a meaningful subgraph in a graph. We use dynamic neighborhood selection to select the most informative nodes for the representation of neighborhood nodes $N_i(t)$ for a given node $v_i(t) \in V$ at a given time $t \in T$. Given a Graph $G = \{v_1, v_2, \dots, v_k\}$ with k nodes, a set of nodes N_i is then a set of closely occurring similar nodes v_i with no overlapping nodes between two given sets of neighbors in the graph. That is, for a graph with M nodes, N_i is a representation of all the neighbors in the graph $G = \bigcup_{i=1}^M N_i$. For two given neighborhood sets N_j and N_k , where $N_j, N_k \in N_i$, $N_j \cap N_k = \emptyset$, for nonoverlapping neighborhood sets $j \neq k$ and $i, j, k = 1, 2, \dots, M$. Generically,

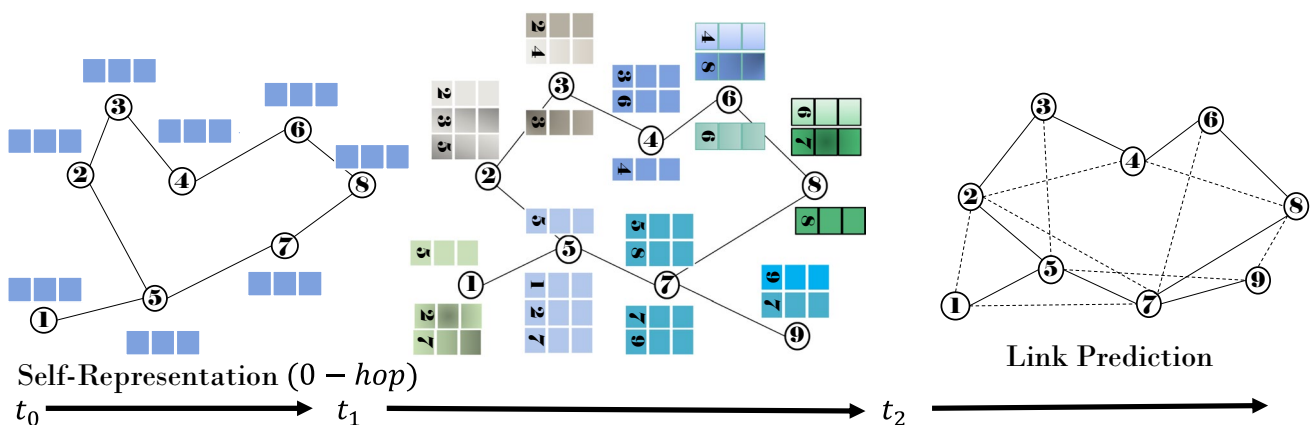


Fig. 2 Left shows the self-representation of each node. Center: Contextual neighborhood representation for each node. The model keeps track of the interactions with nodes at $d - \text{hop}$ level. Nodes with common neighbors are predicted to have a link in the future. $t_0 < t_1 < t_2$ are dynamic time intervals. Example of feature dynamic, event-driven

temporal evolution of node features at unequal time intervals shown by different shades of colors for each node. The boxes show the feature vector of each neighboring node maintained in the form of size-limited tensors. Feature vectors for significant neighbors are concatenated for each target node in the following layers

the feature vector for a neighborhood set N_i , to be represented by node V in the graph, can be defined as shown in Eq. (1).

$$h^{(N_i)} = \frac{1}{M_i} \sum_{k \in S} h_k^p \quad (1)$$

where M_i is the number of nodes, and h_k^p represents the feature vector for each node. To incorporate the temporal evolution model, we consider parallel sub-graph level evolution by capturing combined neighborhood feature adaptation of the nodes significant to a center node with time. This allows us to model the temporal evolution of the framework at different levels of granularity in the graph network. The proposed method assumes that there exists a partially ordered finite set of time points $T = \{t_1, t_2, \dots, t_n\}$, where the time points are in linear order, $t_1 < t_{i+1}$ for $1 < i < n$, and the time intervals are not necessarily of equal length (Fig. 3). The presence or absence of certain node features at different time points can provide useful information in the adaptation of complex systems studying temporal evolution or changes occurring with time. In this paper, for a given set of nodes V in time domain T , the set of dynamic features is represented by H . That is, for each time $t \in T$, we have a dynamic feature set $H(t)$ for the set of nodes V , such that $H(t) = \{h_1(t), h_2(t), \dots, h_n(t)\}$. Now for every node $v \in V$, the feature set h_v^i at time t can be represented as shown in Eq. (2).

$$h_v^i = \frac{1}{M_i} \sum_{k \in V} h_k^p(t) \quad (2)$$

The GNN model for temporal adaptation consists of multiple layers, each of which performs a set of operations on the input data. The layers are connected in a feedforward manner. The input to the model is a graph with nodes representing the data points and edges representing the interactions between them at a given time $G(V, E, T)$. A subgraph $g \subseteq G$ can be constructed for each node using d -hop neighborhood. For $d = 1$, the subgraph is formed with neighbors that are one step away from the given node, and so on. Each subgraph $g \subseteq G$ consists of node v and its d -hop neighbors. Here, a subgraph can be defined as the set of connected nodes in the given time range [45]. The feature vector h_v^d for a node v with neighbors d is updated using the message passing method, where the features of the neighboring nodes are aggregated into the node vector. Thus, after d iterations, a node vector contains information about the node itself as well as its d -hop neighborhood. Once the features are extracted and stored in the nodes, the subgraph g can be passed to a GNN. The node features can be used for performing node or graph classification and link prediction. Generically, as has been defined in [46], the output vector of GNN message passing for the d -th layer can be represented as shown in Eq. (3)

$$h_v^d = C^{(d)}(h_v^{d-1}, AGG^{(d)}(\{h_u^{d-1}, u \in N[v]\})) \quad (3)$$

where C is a combination function, AGG is an aggregation function that aggregates the features of the neighboring nodes, i.e., $\{h_u^{d-1}, u \in N[v]\}$. $N[v]$ is the number of neighboring nodes of the given node v . In the graph, $G(V, E, T)$, each node i has its associated feature vector $h_v^i(t)$ at time $t \in T$. $h_v^i(t-1)$ is the hidden state of node i at the previous time step $t-1$. Each node v_t^i in the graph, G , is connected to an adjacent node v_{t+1}^j through a directed edge e_{ij} . $a_{ij}(t)$ is the attention score of the message from neighbor node j to node i at time t , while W represents the trainable weight matrix. The information for each node is then calculated at the current time step using the message function in Eq. (4)

$$m_{ij}(t) = Relu(W * v_j(t_j), h_i(t-1)) \quad (4)$$

where $m_{ij}(t)$ is the message from neighbor node j to node i at time step t , $Relu$ is the activation function, W is the trainable matrix used to transform the neighbor's features, $v_j(t_j)$ is the feature vector of the neighbor node j at the given time step t_j , which might be different from t . $h_i(t-1)$ captures the historical context. The temporal significance of a message to node i from a neighbor node j at time step t is calculated using the attention score $a_{ij}(t)$. The attention score considers both the message $m_{ij}(t)$ and the historical context $h_i(t-1)$ (Eq. (5)).

$$a_{ij}(t) = softmax(W_a * [m_{ij}(t); h_i(t-1)]) \quad (5)$$

The SoftMax function ensures that the scores sum to 1, representing the relative importance of each neighboring node. W_a is a trainable matrix for the attention mechanism. $[m_{ij}(t); h_i(t-1)]$ is the concatenation of the message $m_{ij}(t)$ and the hidden state $h_i(t-1)$.

Weighted message aggregation $U_i(t)$ for node i at time step t is then used to form the final message received by node i (Eq. (6)).

$$U_i(t) = \sum_{j \in N_i(t)} a_{ij}(t) * m_{ij}(t) \quad (6)$$

Messages from neighboring nodes are weighted based on their corresponding attention scores. The hidden state of the node is updated using a Gated Recurrent Unit (GRU) as shown in Eq. (7). Both the aggregated message $U_i(t)$ and the previous hidden state $h_i(t-1)$ are incorporated into the updated hidden state.

$$h_i(t) = GRU(h_i(t-1), U_i(t)) \quad (7)$$

where $h_i(t)$ is the updated hidden state of node i at time step t . GRU captures the long-term dependencies in the temporal sequence, $h_i(t-1)$ previous hidden state of the node $i(t-1)$, $U_i(t)$ is the aggregated message for node i at time t .

Learnable parameters can be added according to the specific algorithm being used. The CATGNN model can be used for the context-aware temporal adaptation of

an entity in a graph. Train the CATGNN model on the node features, with the objective of predicting whether each node has changed between time posts t_i and t_j , where $t_i, t_j \in T$. To perform link prediction between two given nodes u and v , at time t , with the updated state $h_u(t)$ and $h_v(t)$, $h_{u,v}(t) = [h_u(t) || h_v(t)]$. Multi-layer perceptron (MLP) is then used to predict the link between the two nodes (Eq. (8)).

$$\hat{y}_{uv}(t) = \sigma(MLP(h_{u,v}(t))) \quad (8)$$

MLP is a fully connected neural network consisting of the input, hidden, and output layers, with ReLU as the activation function for the hidden layer and σ as the activation function for the softmax in the output layer. $\hat{y}_{uv}(t) = 0$ shows the absence of an edge between nodes u and v at time t , while a higher value of $\hat{y}_{uv}(t)$ suggests a higher probability of the existence of an edge between nodes. Binary cross entropy loss function L (Eq. (9)) is then used to evaluate the accuracy of the final predictions.

$$L = -\frac{1}{N} \sum_{u,v \in N} y_{uv}(t) (\log(\hat{y}_{uv}(t)) + (1 - y_{uv}(t))) \quad (9)$$

where N is the total number of nodes. Once the model is trained, it is applied to test data at times t_y and $t_z \in T$, using the same graph structure as before. Implementation details are shown in Algorithm 1. First, G is the graph consisting of vertices V and edges, with T representing the set of time stamps that are initialized. A new temporal graph G_T is created where each node g_i from the original graph G is paired with a corresponding time stamp t_i . Dictionaries are created for k nearest neighbors for each subgraph g_i . Features of each node and its k neighbors are combined and aggregated using Eq. (3). Output embeddings are generated o_v for each node. Then, error loss is calculated, and model weights are updated using backpropagation until convergence. Final node embeddings are produced, capturing temporal and contextual information.

Algorithm 1: Dynamic neighborhood selection for link prediction.

Begin

Inputs:

- Graph (G, V, T) , where $t_s > t_e$ showing distinct time stamps
- Maximum neighborhood depth $\leftarrow d$
- Maximum number of neighbors $\leftarrow max_neighbors$

Output:

- Final node embeddings
- Link prediction results

initialize dynamic graph structure $G_T \leftarrow \{(g_i, t_i): g_i \in G, t_i \in T, i = 1, 2, \dots, n\}$

for each pair of nodes v_i, v_j, t in $G_T \leftarrow \{(g_i, t_i): g_i \in G, t_i \in T, i = 1, 2, \dots, n\}$

from 0 to d **do**

for v_i in V at time t_i **do**

initialize the feature tensors $h_{v_i}^d$ **for** k neighbors of v_i with $k \leq max_neighbors$

combine $h_{v_i}^d$ for all non-overlapping neighbors **removing** empty values

update the feature vector **with** the common neighborhood at time t_{i+1}

perform feature aggregation for k neighbors over the $d - hop$ neighborhood

compute message $m_{ij}(t)$ **from** node v_j to v_i at time t

apply attention mechanism to **assign** weights to the messages

aggregate the weighted messages to form the final message $U_i(t)$

for node v_i at time t **do**

update the hidden state for a node v_i

obtain the final embedding **for each** node pair $h_u(t)$ and $h_v(t)$ **do**

concatenate the embeddings of nodes u and v to form a combined representation $[h_u(t) || h_v(t)]$

pass the combined representations through a MLP for link prediction

Return

 final node embeddings and link prediction

End

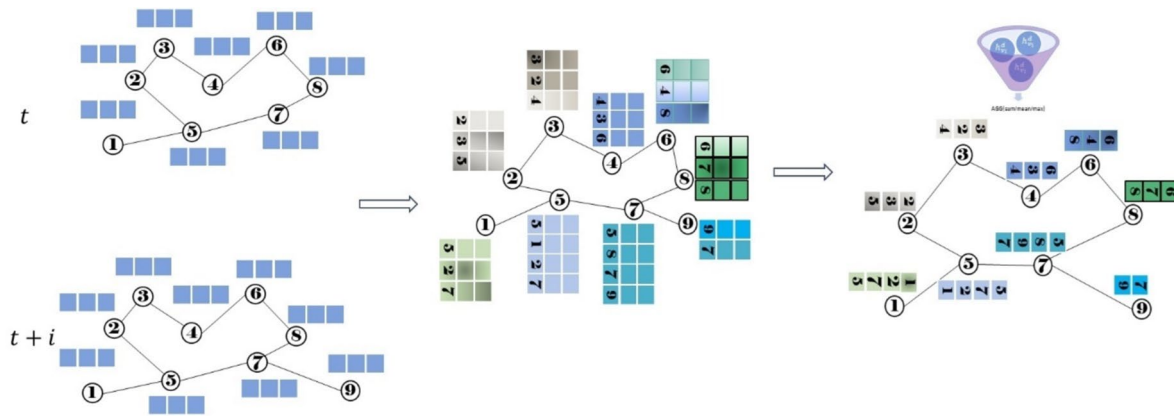


Fig. 3 Feature aggregation from neighboring nodes with contextual temporal adaptation based on common neighborhood. Example illustration shows two subgraphs at subsequent time intervals t and $t+i$ where $i > 0$. Features from d -hop are combined into tensors and are

updated at subsequent intervals. Features are concatenated and aggregated to construct structural feature representations for the subgraph at the given timestep

Experiments and Evaluation

Datasets

Experiments were conducted on six datasets under both inductive and transductive settings. The datasets include Wikipedia, LastFm, Reddit, Enron, Contacts, and Flights. These datasets are publicly available. The datasets were preprocessed in accordance with the standards in previous literature. Table 2 summarizes the statistics of the datasets used in the study, while the details of each dataset are given below:

Reddit Post Dataset [46] the dataset consists of 1-month posts made by users on subreddits. The 1000 most active subreddits were selected as items with 10,000 active users, resulting in 672,447 interactions.

Wikipedia Dataset [46] public dataset with 1 month of edits on Wikipedia pages. 1000 edited pages have been made part of the dataset for experiments as items, with the users making at least five edits, resulting in 8227 users, with 157,474 interactions.

LastFM Dataset [47] dataset consists of users and songs as nodes, and the edges represent the listening behavior of the users. It is a bipartite graph and shows the interactions between the users and songs for over a month. The number of nodes is 1980, and the number of edges is 1,293,103.

Flights Dataset Dataset is a dynamic direct graph network of flights showing air traffic between different airports developed during the COVID-19 pandemic. The dataset was processed and cleaned by [48] and is publicly

available.¹ The tracked flights represent the edges between the airports represented by the nodes. The number of flights between the airports in a day specifies the edge weights.

Enron Dataset [49] consists of 50,000 emails exchanged between employees of ENRON Energy company over a period of 3 years.

Contact Dataset dataset made available² by [48] shows the temporal evolution of four 4-week time period of physical proximity between around 700 university students. The participants are assigned unique IDs, and the edges represent the closeness of proximity between the participants. The physical proximity between the participants is indicated by the edge weights.

These datasets were selected based on the evolving user behavior in their interactions with the items. In the Wikipedia dataset, 79% of interactions consist of users interacting with the same item, while in 61% of interactions in the Reddit dataset, users interact with the same item consecutively.

Hardware The experiments for the evaluation of the proposed model were performed using an Intel Core i7-9850H CPU @ 2.60 GHz with 64.0 GB (63.8 GB usable) and one GPU (NVIDIA Quadro RTX 3000) with 6 GB capacity running a 64-bit Windows 11 Pro operating system. The results from the baseline models were used as have been reported in their original papers.

Evaluation Metrics Evaluating the temporal GNNs is a particularly challenging task, due to the evolving node/edge

¹ <https://zenodo.org/records/7213796#.Y1cO6y8r30o>

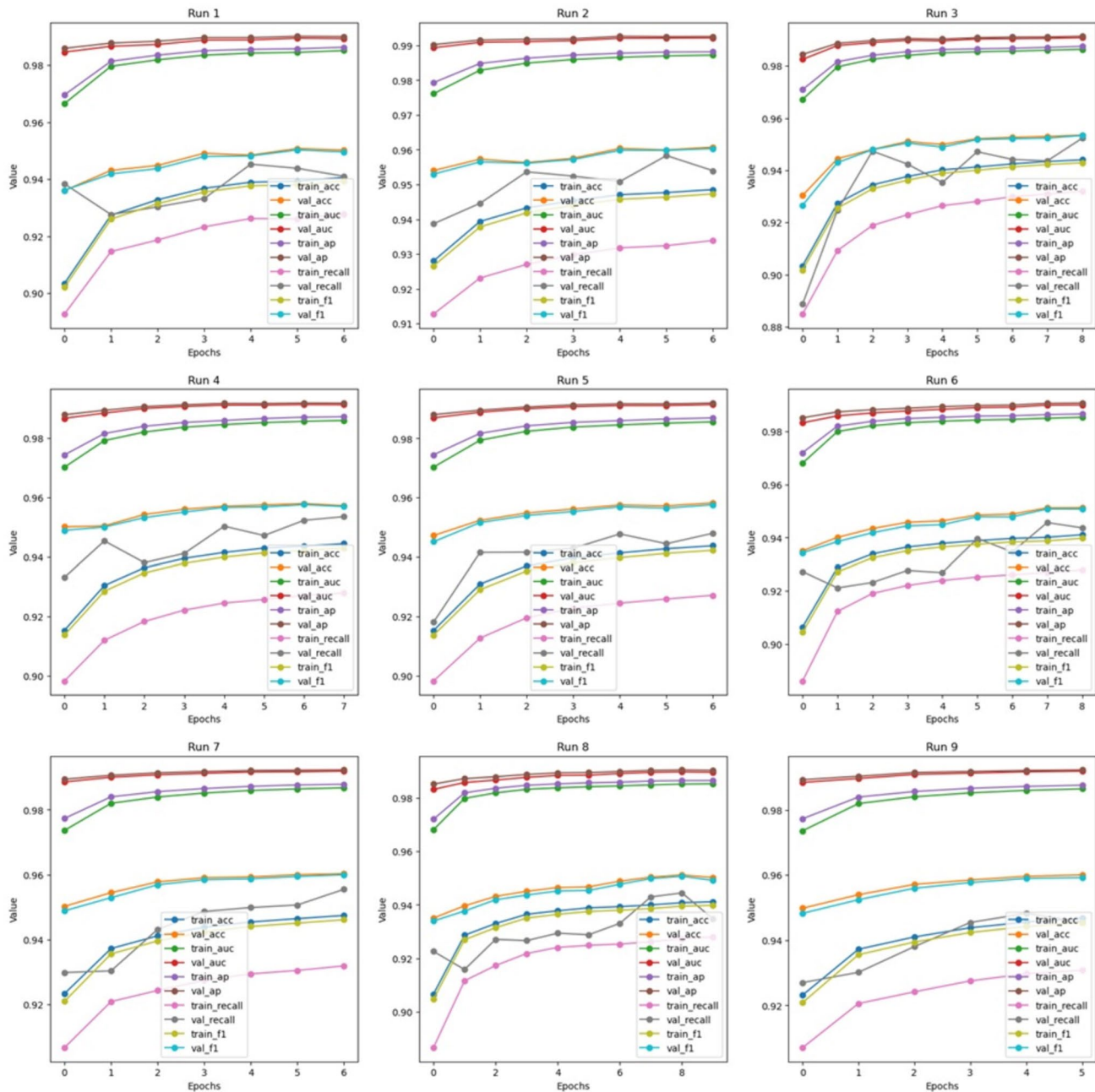
² <https://zenodo.org/records/7213796#.Y1cO6y8r30o>

Table 2 Dataset statistics and features

Dataset	Number of nodes	Number of edges	Time steps	Duration	Domain
Wikipedia	9227	157,474	152,757	1 month	Social media
Redditt	10,984	672,447	669,065	1 month	Social media
LastFM	1980	1,293,103	1,283,614	1 month	Interaction
Enron	184	125,235	22,632	3 years	Social
Flights	13,169	1,947,145	122	4 months	Transport
Contact	694	2,426,280	8065	1 month	Proximity

or attribute features [46]. Experiments were performed on publicly available benchmark datasets to evaluate the

performance of the proposed CATGNN model. The proposed model was evaluated against state-of-the-art baselines using

**Fig. 4** Training and validation on Reddit dataset

standard evaluation measures like Average Precision (AP) and Area Under ROC curve (AUC). Some of the models have been evaluated on AP alone, and some have reported the evaluation on both AP and AUC. For the proposed model, the training and validation accuracy based on hyperparameter tuning has been reported. For evaluation of the computational performance of the model, the average training validation and testing time, the overall run time of the model, and the number of epochs required for convergence from training through validation and testing have been reported.

Hyperparameter Tuning Is a process of selecting appropriate optimal values for parameters during the training of the model. Hyperparameters are configurations, like learning rate, batch size, drop out, number of epochs, time dimension, attention head, bias, etc. that regulate how the model learns. To design the proposed model, appropriate values for hyperparameters are selected based on multiple trials for model optimization. Different values are assigned to these parameters during the model's training process, but the values that yield the best performance on a particular task are chosen as

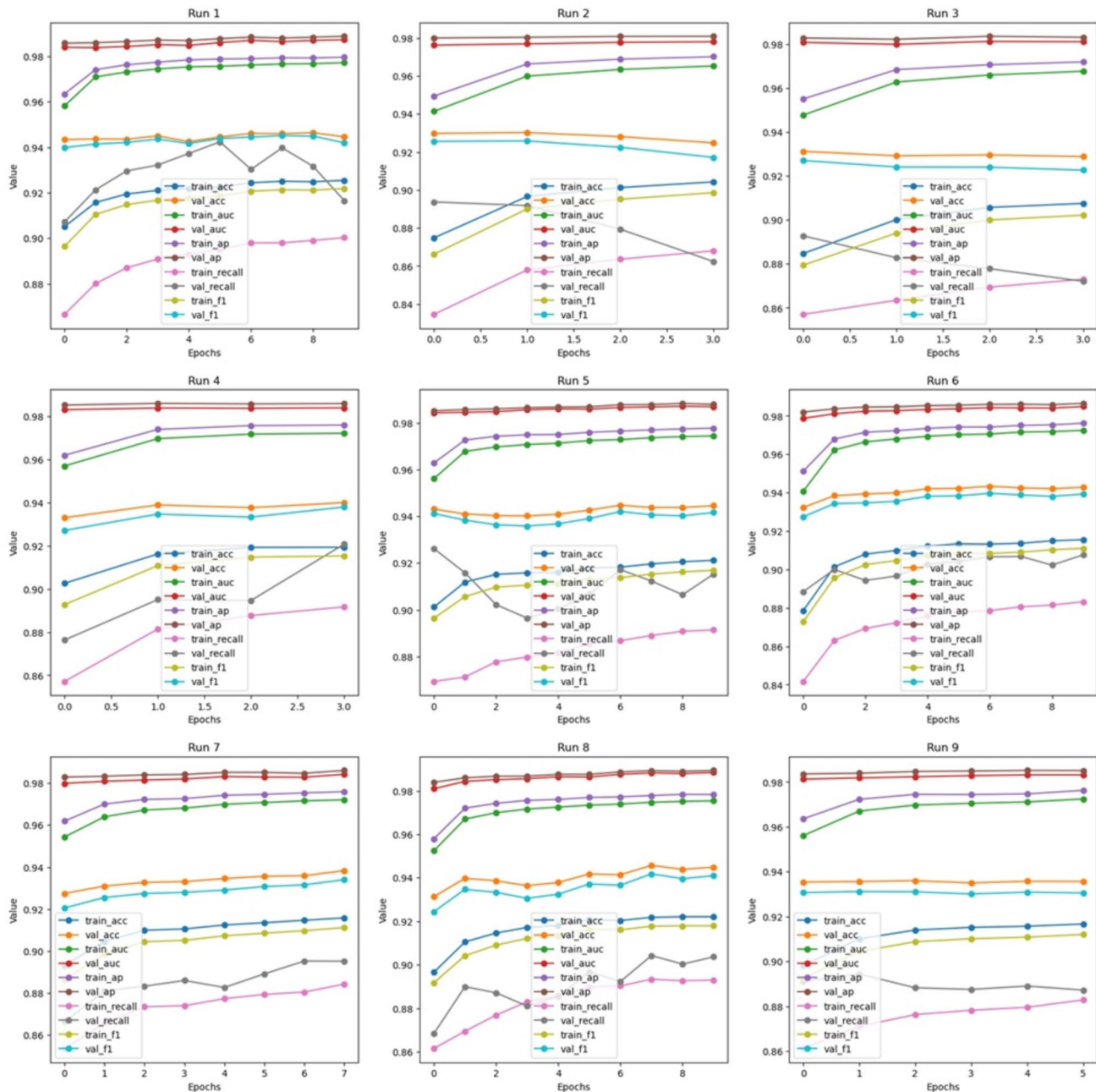
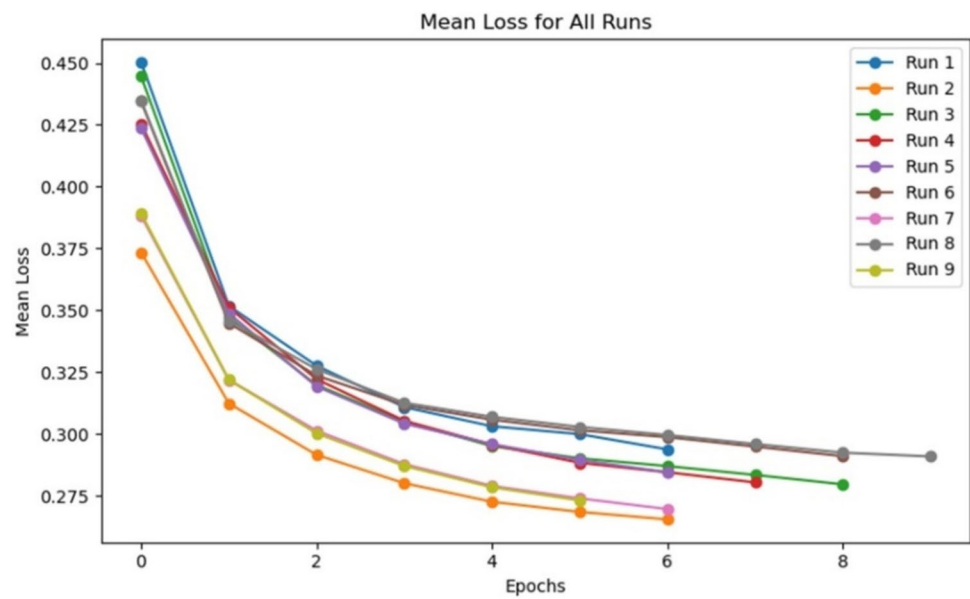
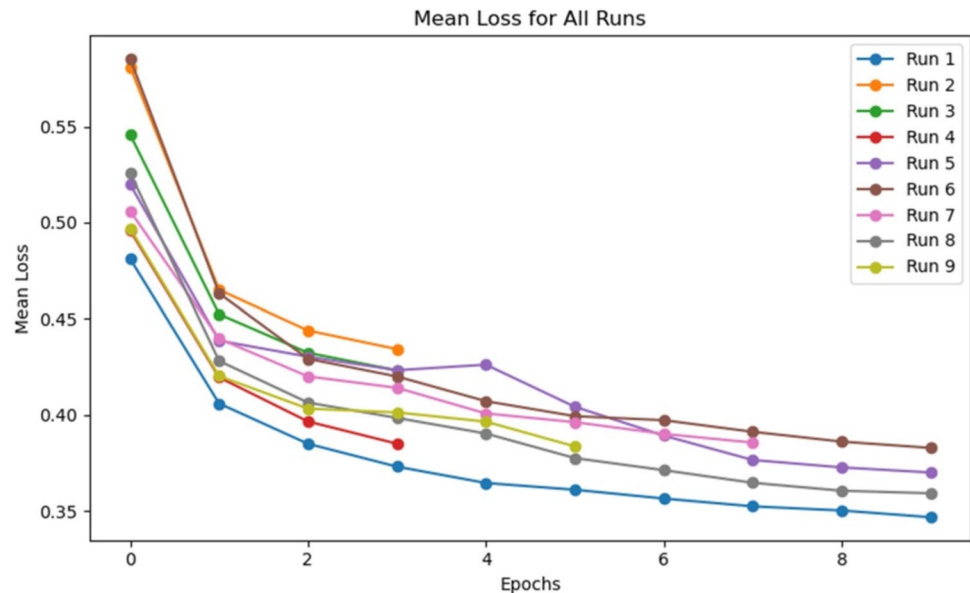


Fig. 5 Training and validation on the Wikipedia dataset

Fig. 6 Mean loss Reddit dataset**Fig. 7** Mean loss per Epoch for each run—Wikipedia dataset

follows: batch size = 100, positional embedding = 16, drop-out = 0.1, with number of attention heads = 2.

Results and Discussion

Experiments were performed on publicly available benchmark datasets to evaluate the performance of the proposed CATGNN model. The proposed model was evaluated against state-of-the-art baselines using standard evaluation measures like accuracy, recall, AUC, average precision, and f1-score. Two real-world datasets, Wikipedia, and Reddit have been used to evaluate the performance of the proposed model. These datasets are publicly available.

Performance on Training and Validation Sets

The CATGNN model was evaluated for convergence and efficiency and demonstrated to achieve optimal results with early convergence across multiple runs. Figure 4 shows the results for training accuracy, AUC, recall, average precision, and f1-score of 0.94, 0.98, 0.93, 0.98, and 0.94, respectively, and the validation scores for accuracy, AUC, recall, average precision, and f1-score of 0.96, 0.98, 0.94, 0.99, and 0.95, respectively, on the Reddit dataset. A similar pattern can be seen for the Wikipedia dataset (Fig. 5), with training accuracy, AUC, recall, average precision, and f1-score of 0.918, 0.98, 0.89, 0.90, 0.98, and 0.91, respectively, and validation

scores for accuracy, AUC, recall, average precision, and f1-score of 0.93, 0.98, 0.89, 0.99, and 0.93, respectively. This indicates that the proposed model is learning effectively as epochs progress. The training and validation results show that CATGNN demonstrates robust performance through all the iterations on both Reddit and Wikipedia datasets.

Mean Loss Score

This section demonstrates the mean loss score of the proposed model on both datasets, including Reddit and Wikipedia datasets, and can be seen in Figs. 6 and 7, respectively. The mean loss represents the average prediction error. Figure 6 shows that the proposed model gradually decreases from an initial value of 0.389 to a final value of 0.273. This loss score indicates consistent improvements in the proposed CATGNN's accuracy over eight epochs using the Reddit dataset. Additionally, Fig. 7 depicts the mean loss of the proposed CATGNN model using the Wikipedia dataset. The suggested CATGNN model gradually drops from an initial value of 0.485 to a final value of 0.310, achieving a mean loss across eight epochs over run 1.

Ablation Study

Performance on Neighborhood Selection

This section demonstrates the performance of neighborhood selection of the proposed CATGNN model. Utilizing Wikipedia datasets for 0-hop, 1-hop, and 2-hop computations, Fig. 8 shows the performance of the suggested CATGNN model in neighborhood selection. The neighborhood selection performance is calculated using standard evaluation metrics, including accuracy, AUC, average precision, recall, and f1-score. In addition, the total time for 0-hop, 1-hop, and 2-hop is calculated. The proposed CATGNN achieved the highest accuracy, AUC, recall, average precision, and f1-score for 1-hop and 2-hop as compared to 0-hop.

Performance on Test Set

The comparative performance of the model on the test set is explained in Section 4.7. This section shows the efficiency of the model in terms of time for two datasets, Reddit and Wikipedia. Figure 9a exhibits the performance of the CATGNN model in terms of accuracy, recall, AUC, average precision, and f1-score trained on a Reddit dataset across ten runs. Figure 9b illustrates the total time spent on the Reddit dataset over ten runs, with time decreasing with each run. Across the ten runs, test accuracy scores range from 90.1603% to 94.507%, showing a high level of efficiency and consistency on Reddit data. The average accuracy across all runs stands at 93.0308%. Test AUC scores range from

97.3229 to 98.9897%, showing a high level of efficiency and consistency on Reddit data. The test average precision scores range from 97.3549 to 99.1304%, highlighting a high level of efficiency and consistency on Reddit data (Fig. 9b).

Figure 10a depicts the performance of the CATGNN model for the evaluation metrics on the Wikipedia dataset across ten runs, while Fig. 10b displays the total time over ten runs on the dataset, along with a time decrease for each run. This indicates that CATGNN demonstrates robust performance throughout ten iterations. From the test results, it can be concluded that by propagating information through the graph structure, the model can capture complex patterns of change that may be challenging to identify with conventional techniques (Tables 3, 4, 5).

Comparison with State of the Art

A comparison of the proposed method with the current state of the art is shown in Table 3 through Table 6 in terms of Average Precision and AUC, respectively, for both inductive and transductive settings. Six real-world datasets, as explained in Section 4.1, were used to compare the Average Precision performance of CATGNN with several baselines.

Baselines

The models chosen for comparison are GAT [50], GraphSAGE [51], JODIE [47], temporal attention network (TGAT) [52], TGN [16], TCL [53], and GraphMixer [54].

GAT leverages masked self-attention layers for specifying different weights for different nodes in a neighborhood.

GraphSAGE is a general inductive framework for efficiently generating node embeddings for previously unseen data by leveraging node feature information.

JODIE temporal bipartite network for user-item interactions employing two coupled recurrent neural networks for updating the states of users and items.

TGAT uses self-attention mechanisms to compute the node representations. It aggregates features from each node's temporal-topological neighbors and is also equipped with a time encoding function for capturing temporal patterns.

TGN utilizes a memory function, memory aggregator, and memory updater to maintain and update the evolving memory for each node appearing in an interaction. Temporal representations of nodes are generated by leveraging and embedding modules.

TCL incorporates graph transformers considering both the temporal information and the graph topology for learning node representation. Interdependence between two interacting nodes is modeled using cross-attention operation.

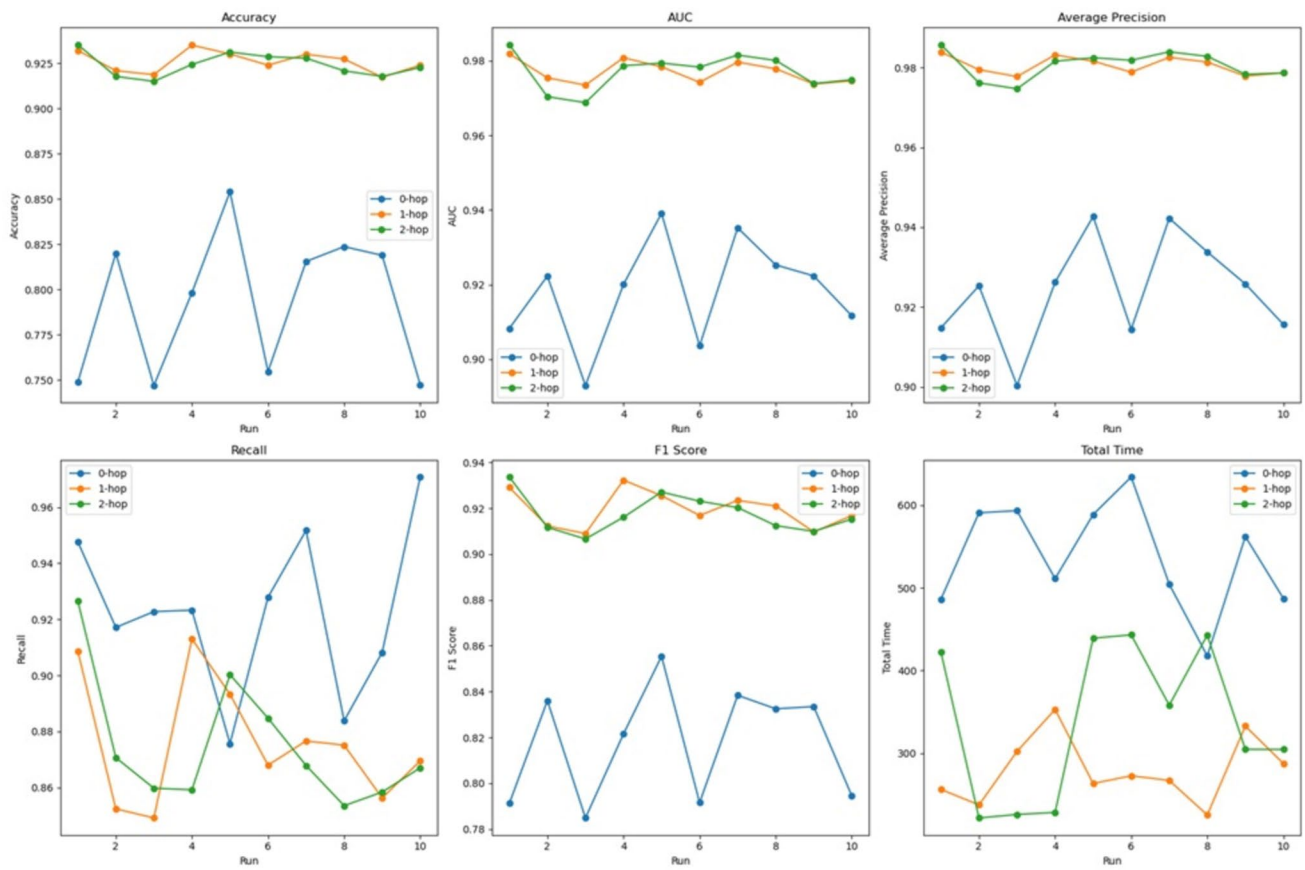


Fig. 8 Neighborhood selection using the Wikipedia dataset

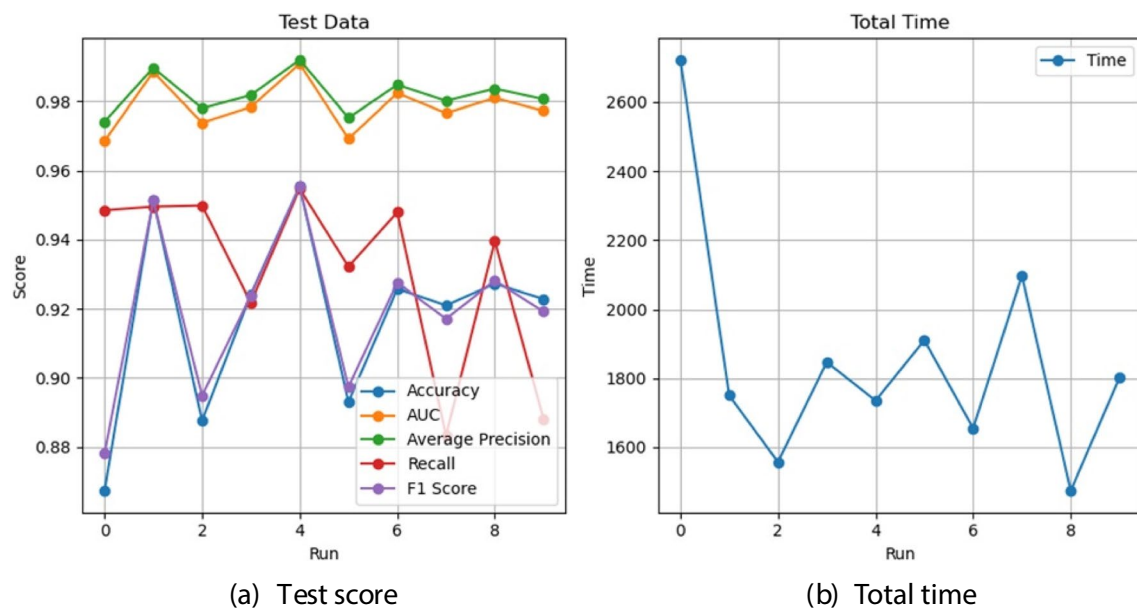


Fig. 9 **a** Test scores and **b** total time for each run on the Reddit dataset

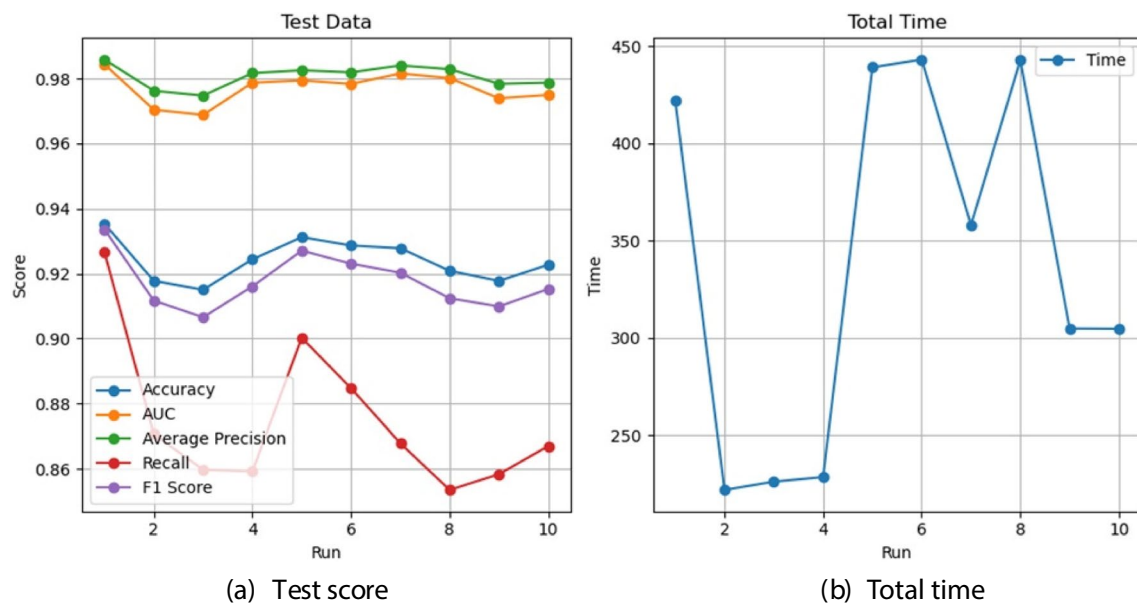


Fig. 10 **a** Test scores and **b** total time for each run on the Wikipedia dataset

Table 3 Comparing the Average Precision of the proposed model with current state-of-the-art graph-based models using inductive settings

Model	Wikipedia	Reddit	Contacts	LastFM	Flights	Enron
GAT	91.27 ± 0.4	95.37 ± 0.3	—	—	—	—
GraphSAGE	91.09 ± 0.3	96.27 ± 0.2	—	—	—	—
JODIE	94.82 ± 0.5	96.50 ± 0.4	94.34 ± 1.45	81.61 ± 3.82	94.71 ± 0.73	80.72 ± 1.39
TGAT	96.22 ± 0.07	97.09 ± 0.04	95.87 ± 0.11	78.63 ± 0.31	88.73 ± 0.33	67.05 ± 1.51
TGN	98.05 ± 0.1	97.71 ± 0.1	93.82 ± 0.99	81.45 ± 4.29	95.03 ± 0.60	77.94 ± 1.02
TCL	96.22 ± 0.17	94.09 ± 0.07	91.11 ± 0.12	73.53 ± 1.66	83.41 ± 0.07	76.14 ± 0.79
GraphMixer	96.65 ± 0.02	95.26 ± 0.02	90.59 ± 0.05	82.11 ± 0.42	83.03 ± 0.05	78.55 ± 0.48
CATGNN	98.62 ± 0.01	98.97 ± 0.01	95.99 ± 0.02	88.02 ± 2.65	97.19 ± 0.01	92.81 ± 1.61

The bold data in the tables indicate the best results obtained for the given dataset

GraphMixer employs a node encoder with neighboring mean-pooling for summarizing node features along with a fixed time encoder function showing that it performs better than the trainable versions.

For the baseline models, we have used the results obtained either from their original papers or from the best variants reported in the literature [38]. For comparison with the baselines, we report the results on two evaluation metrics, Average Precision (AP) and Area Under the Receiver Operating Characteristic Curve (AUC-ROC), for both the transductive and inductive settings.

All the baseline models perform well on Wikipedia and Reddit datasets. More specifically, TGN performs slightly better than CATGNN on inductive AUC on the Wikipedia dataset and is very close to the Reddit dataset. Similarly, TGAT also performs very close to CATGNN on the

Contact dataset. On the Flights dataset, JODIE and TGN show promising results; however, on LastFM, Flights, and Enron datasets, CATGNN performs significantly better than all the other datasets, showing the importance of contextual correlation between the nodes based on common neighborhood for datasets with longer historical sequences. Down sampling of the neighbors does not seem to negatively affect the performance of the model for these datasets and, at the same time, reduces computational complexity.

Conclusions

In this study, the importance of using dynamic neighborhood selection using common neighbors based on attention scores of significant neighbors for a given node is explored.

Table 4 Comparing the AUC of the proposed model with current state-of-the-art graph-based models using inductive settings

Model	Wikipedia	Reddit	Contact	LastFM	Flights	Enron
JODIE	94.33 ± 0.27	96.52 ± 0.16	95.37 ± 0.92	81.13 ± 3.39	95.21 ± 0.32	81.96 ± 1.34
TGAT	95.90 ± 0.09	96.98 ± 0.04	96.53 ± 0.10	76.99 ± 0.29	88.64 ± 0.35	64.63 ± 1.74
TGN	97.84 ± 0.06	97.63 ± 0.09	94.84 ± 0.75	82.61 ± 3.15	95.92 ± 0.43	78.83 ± 1.11
TCL	95.57 ± 0.20	93.80 ± 0.07	93.05 ± 0.09	70.84 ± 0.85	82.48 ± 0.01	72.33 ± 0.99
GraphMixer	96.30 ± 0.14	94.97 ± 0.05	92.83 ± 0.05	80.37 ± 0.18	82.27 ± 0.06	76.51 ± 0.71
CATGNN	97.70 ± 0.01	97.86 ± 0.08	96.64 ± 1.50	86.20 ± 3.71	96.70 ± 1.07	92.41 ± 0.01

The bold data in the tables indicate the best results obtained for the given dataset

Table 5 Comparing the Average Precision of the proposed model with current state-of-the-art graph-based models using transductive settings

Model	Wikipedia	Reddit	Contact	LastFM	Flights	Enron
GAT	94.73 ± 0.1	97.33 ± 0.2	—	—	—	—
GraphSAGE	93.56 ± 0.3	97.65 ± 0.2	—	—	—	—
JODIE	94.33 ± 0.4	96.44 ± 0.14	95.31 ± 1.33	70.85 ± 2.13	95.60 ± 1.73	84.77 ± 0.30
TGAT	95.34 ± 0.1	98.12 ± 0.2	96.28 ± 0.09	73.42 ± 0.21	94.03 ± 0.18	71.12 ± 0.97
TGN	98.64 ± 0.1	98.80 ± 0.1	96.89 ± 0.56	77.07 ± 3.97	97.95 ± 0.14	86.53 ± 1.11
TCL	96.47 ± 0.16	97.53 ± 0.02	92.44 ± 0.12	67.27 ± 2.16	91.23 ± 0.02	79.70 ± 0.71
GraphMixer	97.25 ± 0.03	97.31 ± 0.01	91.92 ± 0.03	75.61 ± 0.24	90.99 ± 0.05	82.25 ± 0.16
CATGNN	98.74 ± 0.01	99.29 ± 0.01	97.38 ± 0.01	90.17 ± 1.9	98.95 ± 0.7	92.60 ± 0.26

The bold data in the tables indicate the best results obtained for the given dataset

Table 6 Comparing the AUC of the proposed model with current state-of-the-art graph-based models using transductive settings

Model	Wikipedia	Reddit	Contact	LastFM	Flights	Enron
JODIE	96.33 ± 0.07	98.31 ± 0.05	96.66 ± 0.89	70.49 ± 1.66	96.21 ± 1.42	87.96 ± 0.52
TGAT	96.67 ± 0.07	98.47 ± 0.02	96.95 ± 0.08	71.59 ± 0.18	94.13 ± 0.17	68.89 ± 1.10
TGN	98.37 ± 0.07	98.60 ± 0.06	97.54 ± 0.35	78.47 ± 2.94	98.22 ± 0.13	88.32 ± 0.99
TCL	94.84 ± 0.18	97.42 ± 0.02	94.15 ± 0.09	64.06 ± 1.16	91.21 ± 0.02	75.74 ± 0.72
GraphMixer	96.92 ± 0.03	97.17 ± 0.02	93.94 ± 0.02	73.53 ± 0.14	91.13 ± 0.01	84.38 ± 0.21
CATGNN	98.60 ± 0.3	99.18 ± 0.2	97.93 ± 0.08	88.95 ± 2.4	98.66 ± 0.7	93.45 ± 0.14

The bold data in the tables indicate the best results obtained for the given dataset

For this, a CATGNN-based method for modeling temporal adaptation in complex networks is presented in this paper. A dynamic neighborhood selection method is used to aggregate messages from significant node neighbors at a given time using attention scores. Nodes that are highly relevant to a node are used for passing messages while temporally insignificant nodes are discarded. This increases the efficiency of the proposed method by using the overall neighborhood of a node for message passing and message aggregation. The CATGNN method is evaluated against the state of the art and shows comparable performance both in terms of accuracy and efficiency. The proposed CATGNN outperforms the baseline on the majority of the datasets and has obtained exceptionally high average precision scores with exceptionally low standard deviations on some datasets, representing high consistency and accuracy in link prediction tasks.

Modeling temporal adaptation in dynamic networks is a significant challenge, and this work focuses on developing a generic model with improved efficiency by using dynamic neighborhood selection for message passing between nodes through the different layers of a GNN. Six real-world datasets have been used to evaluate the performance of the proposed model. The study has implications in diverse fields involving complex systems, such as traffic forecasting, recommender systems, sentiment analysis, climate change detection, citation analysis, social network analysis, etc.

We have performed a comparison of the model with 1-hop and 2-hop depths for neighborhood selection, and it shows a very slight difference. In the future, we intend to perform further ablation studies on the optimal size of neighborhood selection and the effect of temporal encoding. Moreover, since the proposed model exploits the historical

contextual correlations between the nodes based on common neighborhoods, it performs excellently on datasets with longer historical sequences. The performance of datasets with shorter historical contexts might not be at the same level and needs to be optimized and evaluated further.

Author Contribution M. Ali.Zeb; conceptualization, implementation, experimentation, writing the manuscript. M. Irfan Uddin; conceptualization, visualization, critical analysis, mathematical formulation, writing manuscript. A. Abdulsalam Alarood; Mathematical analysis, supervision, reviewing manuscript. M. Shafiq.; Experimentation, resources, software, supervision, reviewing manuscript. S. Habibullah.; Visualization, Analysis, Resources, supervision, manuscript correction. A. A. Alsulami.; Conceptualization, visualization, Mathematical Analysis, Review manuscript.

Data Availability The datasets used in this study are publicly available and can be accessed through the links and references provided in relevant sections in the paper. The project number HU24K14859 Hosei University Japan is acknowledged.

Declarations

Conflict of Interest The authors declare no competing interests.

References

- Chen Q, Jiang F, Guo X, Chen J, Sha K, Wang Y. Combine temporal information in session-based recommendation with graph neural networks. *Expert Syst Appl*. 2024;238:121969. <https://doi.org/10.1016/j.eswa.2023.121969>.
- del Mondo G, Stell JG, Claramunt C, Thibaud R. A graph model for spatio-temporal evolution. *J Univers Comput Sci*. 2010;16(11):1452–77.
- Li Z, Wang C, Wang X, Chen Z, Li J. HJE: joint convolutional representation learning for knowledge hypergraph completion. *IEEE Trans Knowl Data Eng*. 2024;36(8):3879–92. <https://doi.org/10.1109/TKDE.2024.3365727>.
- Gong Y, Lv X, Yuan Z, You X, Hu F, Chen Y. GNN-based multi-modal named entity recognition. *Comput J*. 2024;67(8):2622–32. <https://doi.org/10.1093/comjnl/bxae030>.
- Li H, Li C, Feng K, Yuan Y, Wang G, Zha H. Robust knowledge adaptation for dynamic graph neural networks. *IEEE Trans Knowl Data Eng*. 2024;36(11):6920–33. <https://doi.org/10.1109/TKDE.2024.3388453>.
- Shabani N, et al. A comprehensive survey on graph summarization with graph neural networks. *IEEE Trans Artif Intell*. 2024;5(8):3780–800. <https://doi.org/10.1109/TAI.2024.3350545>.
- Cai H, Zheng VW, Chang KC-C. A comprehensive survey of graph embedding: problems, techniques, and applications. *IEEE Trans Knowl Data Eng*. 2018;30(9):1616–37. <https://doi.org/10.1109/TKDE.2018.2807452>.
- Liang F, Qian C, Yu W, Griffith D, Golmie N. Survey of graph neural networks and applications. *Wirel Commun Mob Comput*. 2022;2022:1–18. <https://doi.org/10.1155/2022/9261537>.
- Barros CDT, Mendonça MRF, Vieira AB, Ziviani A. A survey on embedding dynamic graphs. *ACM Comput Surv*. 2021;55(1):1–37. <https://doi.org/10.1145/3483595>.
- Thomas JM, Moallem-Oureh A, Beddar-Wiesing S, Holzhueter C. Graph neural networks designed for different graph types: A survey. *Trans Mach Learn Res*. 2023. <https://doi.org/10.48550/arXiv.2204.03080>.
- AlBadani B, Shi R, Dong J, Al-Sabri R, Moctard OB. Transformer-based graph convolutional network for sentiment analysis. *Appl Sci*. 2022;12(3):1316. <https://doi.org/10.3390/app12031316>.
- Zhang F, Zheng W, Yang Y. Graph convolutional network with syntactic dependency for aspect-based sentiment analysis. *Int J Comput Intell Syst*. 2024;17(1):37. <https://doi.org/10.1007/s44196-024-00419-6>.
- Allamanis M. Graph neural networks in program analysis. In: Wu L, Cui P, Pei J, Zhao L, editors. *Graph neural networks: foundations, frontiers, and applications*. Singapore: Springer Nature Singapore; 2022. p. 483–497. https://doi.org/10.1007/978-981-16-6054-2_22.
- Carbonell M, Riba P, Villegas M, Fornes A, Lladós J. Named entity recognition and relation extraction with graph neural networks in semi structured documents. In *25th International Conference on Pattern Recognition (ICPR)*. Milan, Italy: IEEE; 2021. p. 9622–9627. <https://doi.org/10.1109/ICPR48806.2021.9412669>.
- Fan W, et al. Graph neural networks for social recommendation,” In *The World Wide Web Conference*. In WWW '19. New York, NY, USA: Association for Computing Machinery; 2019. p. 417–426. <https://doi.org/10.1145/3308558.3313488>.
- Rossi E, Chamberlain B, Frasca F, Eynard D, Monti F, Bronstein M. Temporal graph networks for deep learning on dynamic graphs. In *37th International Conference on Machine Learning Workshop on Graph Representation Learning and Beyond*. Vienna, Austria; 2020.
- Chakraborty A. Aspect based sentiment analysis using spectral temporal graph neural network. In *Proceedings of the 18th International Conference on Natural Language Processing (ICON)*, National Institute of Technology Silchar. Silchar, India: NLP Association of India (NLP AI); 2021. p. 508–518.
- Lu J, Shi L, Liu G, Zhan X. Dual-channel edge-featured graph attention networks for aspect-based sentiment analysis. *Electronics*. 2023;12(3):624. <https://doi.org/10.3390/electronics12030624>.
- Diao Z, Wang X, Zhang D, Liu Y, Xie K, He S. Dynamic spatial-temporal graph convolutional neural networks for traffic forecasting. *Proc AAAI Conf Artif Intell*. 2019;33(01):890–7. <https://doi.org/10.1609/aaai.v33i01.3301890>.
- Li J, et al. Evaluating graph neural networks for link prediction: current pitfalls and new benchmarking. In: Oh A, Naumann T, Globerson A, Saenko A, Hardt M, Levine S, editors. *Advances in neural information processing systems*. Curran Associates, Inc.; 2023. p. 3853–3866. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/0be50b4590f1c5fd4c8fedd63c4f67-Paper-Datasets_and_Benchmarks.pdf
- Bai T, Zhang Y, Wu B, Nie JY. Temporal graph neural networks for social recommendation. In *IEEE International Conference on Big Data*. Atlanta, GA, USA: IEEE; 2020. p. 898–903. <https://doi.org/10.1109/BigData50022.2020.9378444>.
- Fan Z, Liu Z, Zhang J, Xiong J, Zheng L, Yu PS. Continuous-time sequential recommendation with temporal graph collaborative transformer. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. Virtual Event Queensland Australia: ACM; 2021. p. 433–442. <https://doi.org/10.1145/3459637.3482242>.
- Guo J, Han Z, Zhou S, Li J, Tresp V, Wang Y. Continuous temporal graph networks for event-based graph data. In *Proceedings of the 2nd Workshop on Deep Learning on Graphs for Natural Language Processing*. Seattle, Washington: Association for Computational Linguistics; 2022. p. 22–29. <https://doi.org/10.18653/v1/2022.dlg4nlp-1.3>.
- Bhattacharya U, Mittal T, Chandra R, Randhavane T, Bera A, Manocha D. STEP: spatial temporal graph convolutional networks for emotion perception from gaits. *Proc AAAI Conf Artif Intell*. Apr.2020;34(02):1342–50. <https://doi.org/10.1609/aaai.v34i02.5490>.

25. Liu Z, Wang Y, Wang S, Zhao X, Wang H, Yin H. Heterogeneous graphs neural networks based on neighbor relationship filtering. *Expert Syst Appl*. 2024;239:122489. <https://doi.org/10.1016/j.eswa.2023.122489>.
26. Cao D, et al. Spectral temporal graph neural network for multivariate time-series forecasting. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, in NIPS'20. Red Hook, NY, USA: Curran Associates Inc.; 2020.
27. Peng H, Zhang R, Dou Y, Yang R, Zhang J, Yu PS. Reinforced neighborhood selection guided multi-relational graph neural networks. *ACM Trans Inf Syst*. 2021;40(4):1–46. <https://doi.org/10.1145/3490181>.
28. Zang X, Tang B. Self-supervised Dynamic Graph Embedding with evolutionary neighborhood and community. *Expert Syst Appl*. 2023;228:120409. <https://doi.org/10.1016/j.eswa.2023.120409>.
29. Zhu W, Ruan K, Huang J, Xiao J, Yu W. Dynamic graph representation based on temporal and contextual contrasting. In *Proceedings of the 2022 5th International Conference on Algorithms, Computing and Artificial Intelligence*. Sanya China: ACM; 2022, p. 1–8. <https://doi.org/10.1145/3579654.3579771>.
30. Zhang M, Wu S, Gao M, Jiang X, Xu K, Wang L. Personalized graph neural networks with attention mechanism for session-aware recommendation. *IEEE Trans Knowl Data Eng*. 2022;34(8):3946–57. <https://doi.org/10.1109/TKDE.2020.3031329>.
31. Li Y, Wu Y, Sun M, Yang B, Wang Y. Learning continuous dynamic network representation with transformer-based temporal graph neural network. *Inf Sci*. 2023;649:119596. <https://doi.org/10.1016/j.ins.2023.119596>.
32. Wang F, Hou D, Yan H. Detecting community evolution by utilizing individual temporal semantics in social networks. *IEEE Access*. 2023;11:120362–71. <https://doi.org/10.1109/ACCESS.2023.3328616>.
33. Mu Z, Zhuang Y, Tang S. Contrastive Hawkes graph neural networks with dynamic sampling for event prediction. *Neurocomputing*. 2024;575:127265. <https://doi.org/10.1016/j.neucom.2024.127265>.
34. Gao S, Li Y, Shen Y, Shao Y, Chen L. ETC: efficient training of temporal graph neural networks over large-scale dynamic graphs. *Proc VLDB Endow*. 2024;17(5):1060–72. <https://doi.org/10.14778/3641204.3641215>.
35. Gao S, Li Y, Zhang X, Shen Y, Shao Y, Chen L. SIMPLE: efficient temporal graph neural network training at scale with dynamic data placement. *Proc ACM Manag Data*. 2024;2(3):1–25. <https://doi.org/10.1145/3654977>.
36. Zhang H, Han X, Xiao X, Bai J. Time-aware graph structure learning via sequence prediction on temporal graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. Birmingham United Kingdom: ACM; 2023, p. 3288–3297. <https://doi.org/10.1145/3583780.3615081>.
37. Wu Y, Fang Y, Liao L. On the feasibility of simple transformer for dynamic graph modeling. In *Proceedings of the ACM Web Conference 2024*. Singapore Singapore: ACM; 2024, p. 870–880. <https://doi.org/10.1145/3589334.3645622>.
38. Yu L, Sun L, Du B, Lv W. Towards better dynamic graph learning: new architecture and unified library. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, in NIPS '23. Red Hook, NY, USA: Curran Associates Inc.; 2024.
39. Cong W, et al. DyFormer : a scalable dynamic graph transformer with provable benefits on generalization ability. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*. 2023, p. 442–450. <https://doi.org/10.1137/1.9781611977653.ch50>.
40. Peng H, Zhang R, Dou Y, Yang R, Zhang J, Yu PS. Reinforced neighborhood selection guided multi-relational graph neural networks. *ACM Trans Inf Syst*. 2022;40(4):1–46. <https://doi.org/10.1145/3490181>.
41. Xie Y, Li S, Yang C, Wong RCW, Han J. When do GNNs work: understanding and improving neighborhood aggregation. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, Yokohama, Japan: International Joint Conferences on Artificial Intelligence Organization. 2020, p. 1303–1309. <https://doi.org/10.24963/ijcai.2020/181>.
42. Li Y, Shen Y, Chen L, Yuan M. Zebra: when temporal graph neural networks meet temporal personalized PageRank. *Proc VLDB Endow*. 2023;16(6):1332–45. <https://doi.org/10.14778/3583140.3583150>.
43. Ghafouri S, Khasteh SH, Azarkasb SO. Influence maximization (IM) in complex networks with limited visibility using statistical methods. *J Supercomput*. 2024;80(5):6809–54. <https://doi.org/10.1007/s11227-023-05695-1>.
44. Finkelshtein B, Huang X, Bronstein M, Ceylan II. Cooperative graph neural networks. In: *Proceedings of forty-first International Conference on Machine Learning (ICML)*. 2024. [Online]. Available: <https://arxiv.org/abs/2310.01267>. Accessed 10 Oct 2024.
45. Kovanen L, Karsai M, Kaski K, Kertész J, Saramäki J. Temporal motifs in time-dependent networks. *J Stat Mech Theory Exp*. 2011;2011(11):P11005. <https://doi.org/10.1088/1742-5468/2011/11/P11005>.
46. Longa A, et al. Graph neural networks for temporal graphs: state of the art, open challenges, and opportunities. *Trans Mach Learn Res*. 2023.
47. Kumar S, Zhang X, Leskovec J. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Anchorage AK USA: ACM; 2019, p. 1269–1278. <https://doi.org/10.1145/3292500.3330895>.
48. Poursafaei F, Huang S, Pelrine K, Rabbany R. Towards better evaluation for dynamic link prediction. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2022, p. 32928–32941.
49. Shetty J, Adibi J. The Enron email dataset database schema and brief statistical report. Information sciences institute technical report. University of Southern California; 2004. [Online]. Available: <http://www.cs.cmu.edu/~enron/>. Accessed 26 Aug 2024.
50. Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. Graph attention networks. In: *International Conference on Learning and Representation*. 2018. [Online]. Available: <http://arxiv.org/abs/1710.10903>. Accessed 17 Nov 2023.
51. Hamilton WL, Ying R, Leskovec J. Inductive representation learning on large graphs. In *31st Conference on Neural Information Processing Systems*. CA, USA: Long Beach; 2017.
52. Xu D, Ruan C, Korpeoglu E, Kumar S, Achan K. Inductive representation learning on temporal graphs. Feb. 18, 2020, *arXiv: arXiv:2002.07962*. [Online]. Available: <http://arxiv.org/abs/2002.07962>. Accessed 14 May 2023.
53. Wang L, et al. TCL: Transformer-based dynamic graph modelling via contrastive learning. May 17, 2021, *arXiv: arXiv:2105.07944*. [Online]. Available: <http://arxiv.org/abs/2105.07944>. Accessed 30 Aug 2024.
54. Cong W, et al. Do we really need complicated model architectures for temporal networks? In: *The Eleventh International Conference on Learning Representations*. 2023. [Online]. Available: <https://openreview.net/forum?id=ayPPcOSyL.v1>. Accessed 30 Aug 2024.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.